

RAG Pipeline – Practical Reference Document

Retrieval-Augmented Generation (RAG) pipeline for a PDF-based knowledge assistant

1. What is RAG?

RAG stands for Retrieval-Augmented Generation. It combines information retrieval with a generative Large Language Model (LLM). Instead of asking the LLM to answer only from its pretrained knowledge, the application first retrieves relevant information from an external knowledge base and then provides that information to the LLM as context.

Core idea: User Question → Retrieve Relevant Information → Give Context to LLM → Generate Answer

2. Complete Pipeline

PDF / Documents

↓

Text Extraction

↓

Cleaning / Structuring

↓

Chunking

↓

Embedding Model

↓

Vector Database

User Question

↓

Query Embedding

↓

Vector Search

↓

Top-K Relevant Chunks

↓

Prompt Construction

↓

LLM Generation

↓

Final Answer

3. Document Ingestion Phase

Step 1 – Load the document

The application reads a source such as a PDF. A document loader/parser converts the file into usable text.

Example tools: pdf-parse, PDF loaders, OCR/document AI tools.

Step 2 – Extract text

For text-based PDFs, a parser can extract text directly. Scanned PDFs may require OCR. Complex documents may require layout-aware or multimodal extraction.

Step 3 – Chunk the text

Large documents are split into smaller pieces called chunks. Chunking improves retrieval because the vector database can return the specific portions relevant to a question.

A practical starting point is around 500–1000 tokens or roughly 1000–3000 characters, but the best size depends on the document and retrieval quality. `RecursiveCharacterTextSplitter` is a common general-purpose approach.

Example configuration:

`chunkSize: 1000`

`chunkOverlap: 200`

4. Embeddings

An embedding model converts text into a numerical vector that represents its semantic meaning. Similar pieces of text tend to have vectors that are close to each other in vector space.

In this project, Mistral's embedding model can be used for both document embeddings and query embeddings.

Important: If documents were embedded with one embedding model, the query should normally be embedded with the same model and compatible embedding space.

5. Vector Database – Pinecone

The generated vectors are stored in Pinecone along with metadata such as the original chunk text, document ID, page number, or other useful filters.

Conceptually, a stored record looks like:

`id` → `chunk-001`

`values` → embedding vector

`metadata.text` → original chunk

6. Retrieval Phase

Step 1 – User asks a question

Example: "Explain the machine coding round."

Step 2 – Embed the query

The same embedding model used for the stored documents converts the question into a query vector.

Step 3 – Search Pinecone

Pinecone compares the query vector with stored vectors and returns the most semantically similar chunks.

Step 4 – Select Top-K results

The application commonly retrieves a small number of relevant chunks, such as `topK = 3` or `topK = 5`. The correct value should be evaluated against the quality of the answers.

7. Prompt Construction

The retrieved chunks are combined with instructions and the original user question. This creates the prompt or message structure sent to the generation model.

A useful structure is:

System Instructions

+

Retrieved Context

+

User Question

Example prompt

You are a helpful RAG assistant.

Use the provided context to answer the user's question.

CONTEXT:

[retrieved chunks]

QUESTION:

[user question]

If the answer is not available in the context, say that you do not have enough information.

8. Generation Phase

The retrieved context is passed to a generative LLM. The LLM uses the context and the user's question to produce the final natural-language answer.

The embedding provider and generation provider do not have to be the same. For example:

Mistral → embeddings

Pinecone → retrieval

Gemini → generation

9. Current Project Architecture

Your current architecture can be represented as:

PDF

↓

pdf-parse

↓

RecursiveCharacterTextSplitter

↓

MistralAIEmbeddings (mistral-embed)

↓

Pinecone

User Question

↓
MistralAIEmbeddings.embedQuery()
↓
Pinecone query()
↓
Retrieved Context
↓
LangChain SystemMessage + HumanMessage
↓
ChatGoogleGenerativeAI
↓
Final Answer

10. Example Retrieval Code

```
const vector = await embeddings.embedQuery(userPrompt);

const response = await index.query({
  vector,
  includeMetadata: true,
  topK: 3,
});

const context = response.matches
  .map(match => match.metadata?.text)
  .filter(Boolean)
  .join("\n\n");
```

11. Example Generation Code

```
const ai = new ChatGoogleGenerativeAI({
  apiKey: process.env.GEMINI_API_KEY,
  model: "gemini-3.8-flash",
});

const aiResponse = await ai.invoke([
  new SystemMessage(`
You are a helpful RAG assistant.

Answer using the provided context.

Context:
${context}

If the answer is not available in the context,
say that you don't have enough information.
`),
  new HumanMessage(userPrompt),
]);

console.log(aiResponse.content);
```

12. Why RAG is Used

RAG is useful when the application needs information that may be private, frequently changing, domain-specific, or too large to place directly into every prompt. It can reduce unsupported answers by

grounding generation in retrieved information, but it does not guarantee zero hallucinations.

13. Updating Documents

When a source document changes, its indexed representation should be updated. Production systems commonly track document IDs, versions, timestamps, or content hashes. Old chunks can be removed or replaced and new chunks can be embedded and indexed. Incremental indexing avoids rebuilding the entire knowledge base unnecessarily.

14. Production Improvements

- Use deterministic chunk IDs instead of `Math.random()`.
- Store document ID, page number, section, version, and source metadata.
- Use metadata filtering when multiple documents or tenants exist.
- Evaluate chunk size and overlap using real retrieval tests.
- Consider hybrid search when keyword matching is important.
- Use reranking when initial vector retrieval returns several plausible chunks.
- Add retry/backoff only for transient provider errors such as temporary 429/503 responses.
- Monitor embedding cost, vector count, retrieval quality, latency, and generation cost.
- For large systems, keep original files in object storage and use the vector database for indexed chunks/vectors/metadata.

15. Debugging Checklist

- Check whether PDF text extraction actually produced useful text.
- Print the number of generated chunks.
- Verify document embedding count matches chunk count.
- Verify Pinecone index dimension matches the embedding model.
- Inspect retrieved chunks before calling the LLM.
- Check that retrieved `metadata.text` is present.
- Verify the LLM receives both context and the user's question.
- Distinguish provider authentication errors from quota/rate-limit errors.
- A 429 quota error usually concerns provider limits, not chunking or Pinecone retrieval.

16. Interview Mental Model

Embedding: Converts text into vectors for semantic comparison.

Vector database: Stores and searches vectors plus metadata.

Retrieval: Finds the chunks most relevant to the user's query.

Prompt construction: Combines instructions, retrieved context, and the user's question.

Generation: The LLM uses that context to produce the final response.